CADT IDT

***W5** PRACTICE*

React + Axios Integration

At the end of this practice, you can

- ✓ **Apply** APIs integrations between your existing APIs with ReactJS
- ✓ **Understanding** APIs integration with using different request methods

Get ready before this practice!

- ✓ **Read** the following documents to understand the nature of Express.js: https://expressjs.com/
- ✓ **Read** the following documents to know more about Axios: https://axios-http.com/docs/intro
- ✓ **Read** the following documents to know more about Axios with React: https://www.digitalocean.com/community/tutorials/react-axios-react

How to submit this practice?

- ✓ Once finished, push your **code to GITHUB**
- ✓ Join the **URL of your GITHUB** repository on LMS

EXERCISE 1 – APIs integration for Articles

- *For this exercise you will start with a **START CODE (EX-1)***

Goals

- ✓ Integrate React with RESTful APIs
- ✓ Use Axios to perform GET, POST, PUT, DELETE requests
- ✓ Display and manipulate dynamic data in components
- ✓ Apply controlled components and form handling
- ✓ Understand state, effect hooks, and API-driven routing

Context

You are a frontend developer at the same news company. Your job now is to connect the existing REST API built in Express.js to a React frontend to allow users to:

1. Browse, view, and manage articles
2. Filter articles by journalist or category
3. Create and update content via forms

Q1 – Display All Articles

Task: Build a component that fetches and displays all articles.

API Used: `GET /articles`

Axios REQUEST in ReactJS

`useEffect(() => {`
`  axios.get('http://localhost:5000/articles')`
`  .then(res => setArticles(res.data))`
`  .catch(err => console.error(err));`
`}, []);`

Q2 – View Article Details

Task: Show full content of an article using dynamic route `/articles/:id`.

API Used: `GET /articles/:id`
Use `useParams()` from React Router.

Axios REQUEST in ReactJS
`axios.get(`http://localhost:5000/articles/${id}`)`

Q3 – Add New Article

Task: Create a form to add a new article.

API Used: `POST /articles`
Fields: `title`, `content`, `journalistId`, `categoryId`

Axios POST REQUEST to create an article
`axios.post('http://localhost:5000/articles', articleData)`

Q4 – Update Existing Article

Task: Prefill a form and update an existing article.

API Used: `PUT /articles/:id`

Axios POST REQUEST to update an article
`axios.put(`http://localhost:5000/articles/${id}`, updatedData)`

Reflective Questions

1. How did using `useEffect()` and `axios` help separate logic from UI?

⇨ Using `useEffect()` and `axios` help separate logic from UI because the API request code stayed inside `useEffect()`, while the JSX only focused on displaying data. For example, `axios.get()` handled fetching articles, `setArticles()` stored the result, and the UI simply used `articles.map()` to display them.

2. What state challenges did you face while managing form input and API response?

⇨ Challenges I face while managing form input and API response were keeping the form input updated and handling API response data correctly. Each input needed to update the right property in state, such as `title`, `content`, `journalistId`, and `categoryId`. Another challenge was that input values are strings by default, so IDs sometimes needed to be converted to numbers before sending them to the backend.

3. How does REST structure help React developers write cleaner frontend code?

⇨ REST structure helps React developers write cleaner frontend code because each API endpoint has a clear purpose. For example, `GET /articles` gets all articles, `GET /articles/:id` gets one article, `POST /articles` creates an article, and `PUT /articles/:id` updates one. This makes the React code easier to organize by matching each component or action to a specific API request.

EXERCISE 2 – *API Integration: Filter Articles by Journalist & Category*

- *For this exercise you will start with a **START CODE (EX-2)***

Goals

- ✓ Use dropdown selections (<select>) to trigger filtered API requests
- ✓ Understand sub-resource routes in REST (/journalists/:id/articles, etc.)
- ✓ Dynamically render filtered content
- ✓ Manage dependent state and conditional rendering in React
- ✓ Practice clean UI/UX with form inputs

Context

Your frontend application should now allow users to filter articles based on:

- **A selected journalist**
- **A selected category**

Each filter option will call a different API route and update the displayed list of articles accordingly.
You will not filter client-side, but instead make API requests based on selection.

Q1 – Fetch All Journalists & Categories for Select Inputs

Task: On component mount, fetch journalists and categories to populate two dropdown menus.

APIs Used:

- `GET /journalists`
- `GET /categories`

Axios GET REQUEST to fetch an journalists and categories

```
useEffect(() => {
  axios.get('http://localhost:5000/journalists').then(res =>
setJournalists(res.data));
  axios.get('http://localhost:5000/categories').then(res =>
setCategories(res.data));
}, []);
```

Q2 – Filter Articles by Selected Journalist

Task: When a journalist is selected from the dropdown, fetch articles written by that journalist.

API Used:

- `GET /journalists/:id/articles`

Axios GET REQUEST to fetch articles

```
axios.get(`http://localhost:5000/journalists/${selectedJournalistId}/article
s`)
  .then(res => setArticles(res.data));
```

Q3 – Filter Articles by Selected Category

Task: When a category is selected from the dropdown, fetch articles belonging to that category.

API Used:

Axios GET REQUEST to fetch articles
`axios.get(`http://localhost:5000/journalists/${selectedJournalistId}/articles`)` `  .then(res => setArticles(res.data));`

Q4 – (Bonus) Combine Filters (Frontend Side)

Task: Apply **combined filtering** on the frontend (e.g., articles that match both selected journalist and category) after fetching results from one of the filters.

- Adjust the backend to support combined filtering through query parameters if needed:

```
// Example: GET /articles?journalistId=1&categoryId=2
```

REFLECTIVE QUESTIONS

- *For this part, submit it in separate PDF files*

1. **How do sub-resource routes like `/journalists/:id/articles` help in designing a clear and organized API?**
   Explain the benefits of using nested routes for resource relationships in REST APIs.

⇨ Sub-resource routes such as `/journalists/:id/articles` make APIs more organized by clearly showing the relationship between resources. Instead of getting all articles and filtering them later, the route directly indicates that the requested articles belong to a specific journalist. This improves readability and makes the API easier to understand and maintain.

2. **What challenges did you face when managing multiple filter states (journalist and category) in React?**
   Discuss how you handled state updates and conditional rendering when multiple inputs affect the same output.

⇨ Challenges that I face when managing multiple filter states for journalist and category are that both selections could affect the same list of displayed articles. To handle this, separate state variables were used for each filter, and state updates gets the filtered data. Conditional rendering ensured that the article list updated correctly whenever either filter changed, preventing outdated or incorrect results from being displayed.

3. **What would be the advantages and disadvantages of handling the filtering entirely on the frontend versus using API-based filtering?**
   Compare client-side filtering (in-memory) vs. server-side filtering (via query or nested routes).

⇨ The advantage of handling the frontend filtering is that it's simpler and provides faster interactions because all data is already loaded into memory, reducing the number of API requests. However, its disadvantage is it becomes inefficient when dealing with large datasets since more data must be transferred and stored in the browser. API-based filtering is more scalable because only relevant data is returned from the server, but it requires additional API design and may result in more network requests whenever filter values change.

4. **If you needed to allow filtering by both journalist and category at the same time on the backend, how would you modify the API structure?**
   Think about adding query parameters or designing a new combined route.

⇨ To support filtering by both journalist and category at the same time on the backend, I would add query parameters to a single route such as

`/articles?journalistId=1&categoryId=2.`

This approach is flexible because users can filter by either parameter individually or combine both at the same time. It also avoids creating many separate routes for different filter combinations and keeps the API easier to maintain.

5. **How did this exercise help you understand the interaction between React state, form controls, and RESTful API data?**
   Reflect on how state changes triggered data fetching and influenced the UI rendering logic.

⇨ This exercise improved my understanding of how React state, form controls, and RESTful APIs work together. The selected values from the dropdown menus were stored in state, and changes to that state triggered API requests through Axios. The returned data then updated the component state, causing React to re-render the UI with the filtered articles. This demonstrated how user interactions can drive data fetching and dynamically update the interface.